

АиСД (2025/11/16). SET 3 — Задача A3b. Анализ INTROSORT

1. Реализация Introsort (A3i)

ID посылки в Codeforces: 349354077 (принята)

Репозиторий: https://github.com/AlexanderOsharov/ADS/tree/2025-11-16__SET-3/A3

Файлы:

- `introsort.cpp` — реализация Introsort
- `introsort_data_generator.cpp` — генерация данных (15 запусков)
- `a3_analysis.py` — анализ и графики

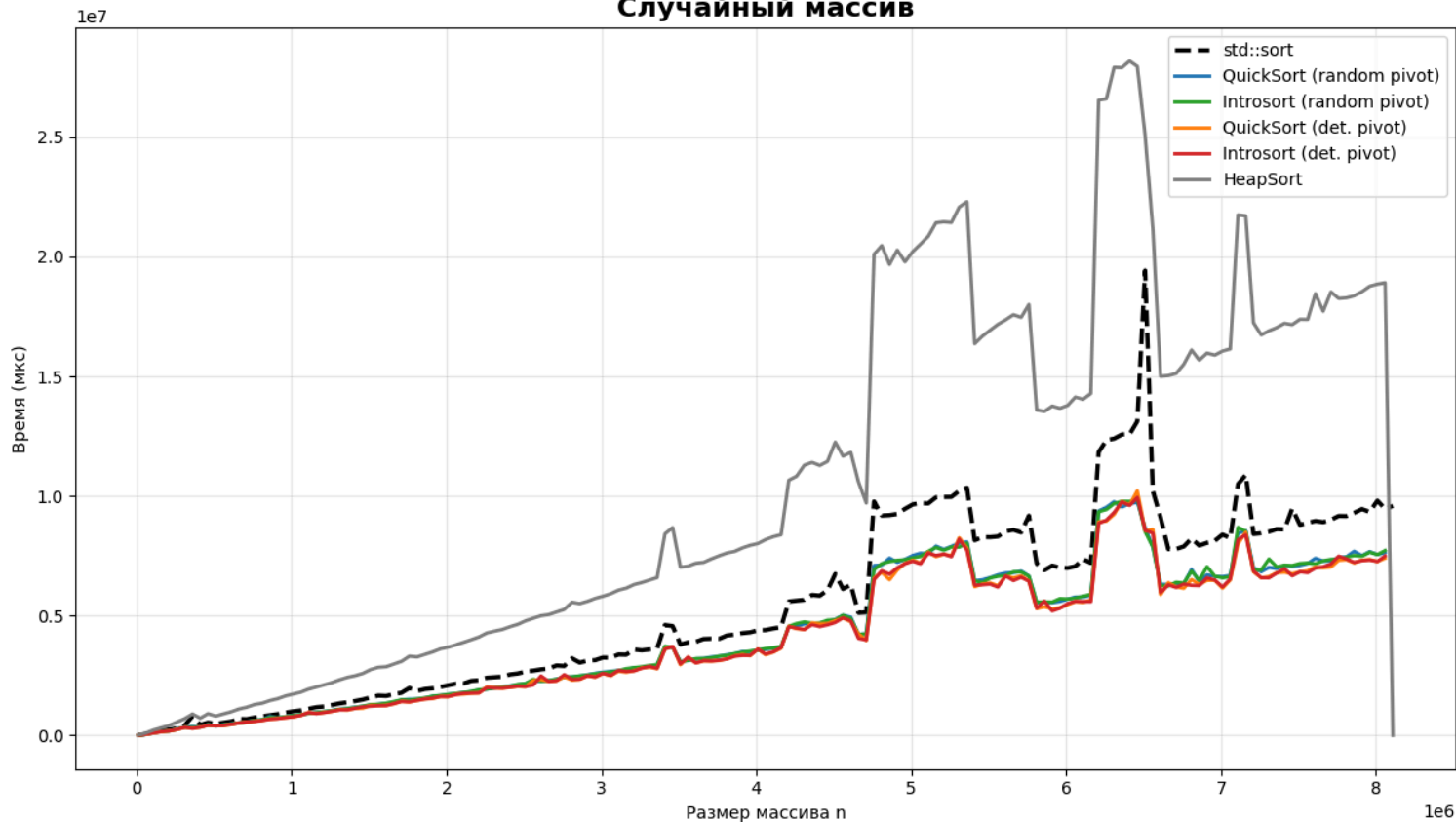
2. Условия эксперимента

Параметр	Значение
Размеры массивов (хорошие случаи)	$n \in [10\,000; 10\,000\,000]$, шаг 50 000
Размеры массивов (плохие случаи)	$n \in [5\,000; 500\,000]$, шаг 10 000
Количество запусков на точку	15
Типы входных данных	Случайный, почти отсортированный, обратно отсортированный, полностью отсортированный
Глубина переключения	$2 \cdot \lfloor \log_2 n \rfloor$ (стандарт)
Попор Insertion Sort	16
Компиляция	<code>g++ -O3 -march=native</code>

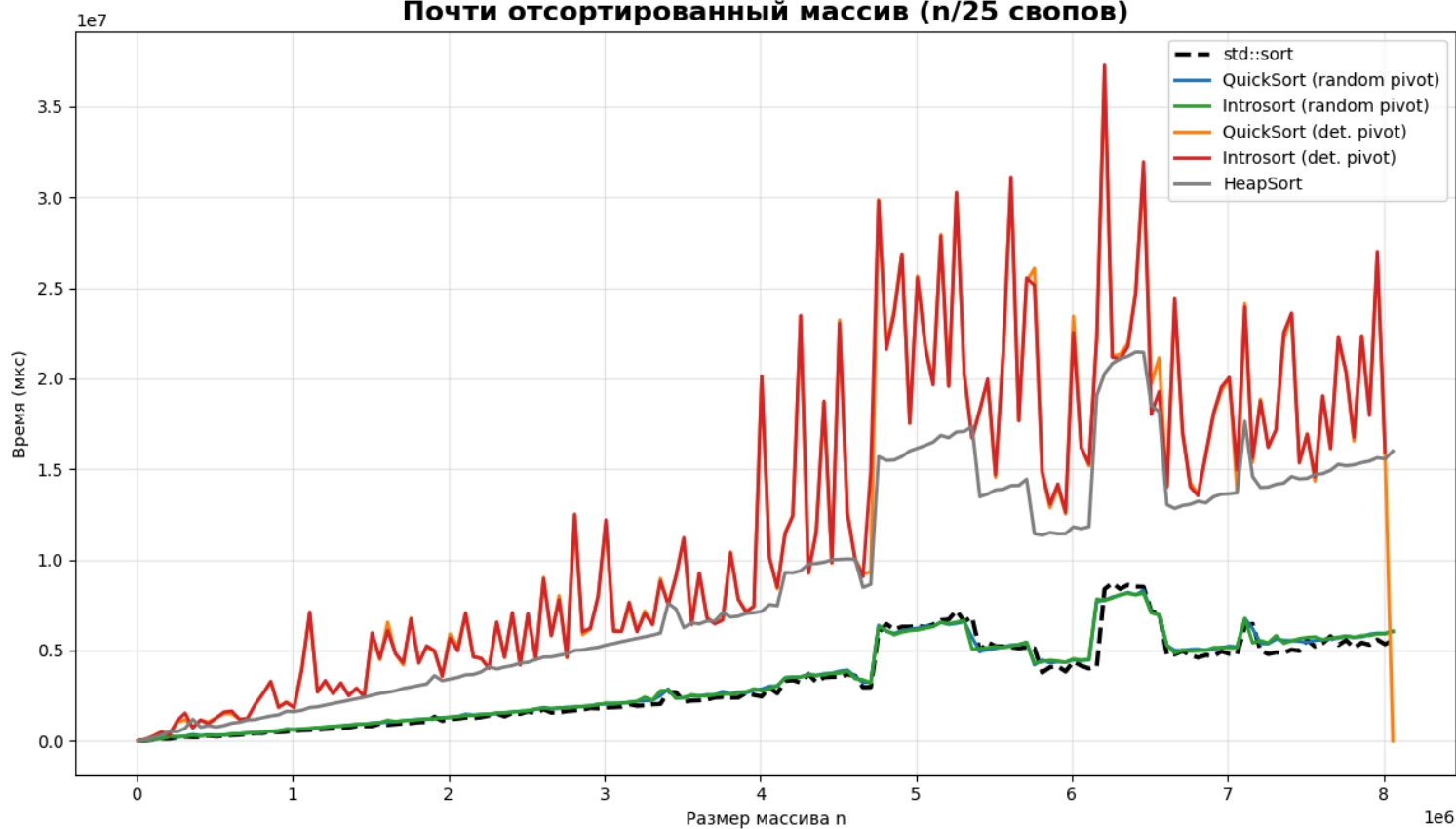
3. Результаты и анализ

3.1. Случайный и почти отсортированный массивы

Случайный массив



Почти отсортированный массив (n/25 свопов)



Обратно отсортированный массив (плохие случаи)



Полностью отсортированный массив (плохие случаи)



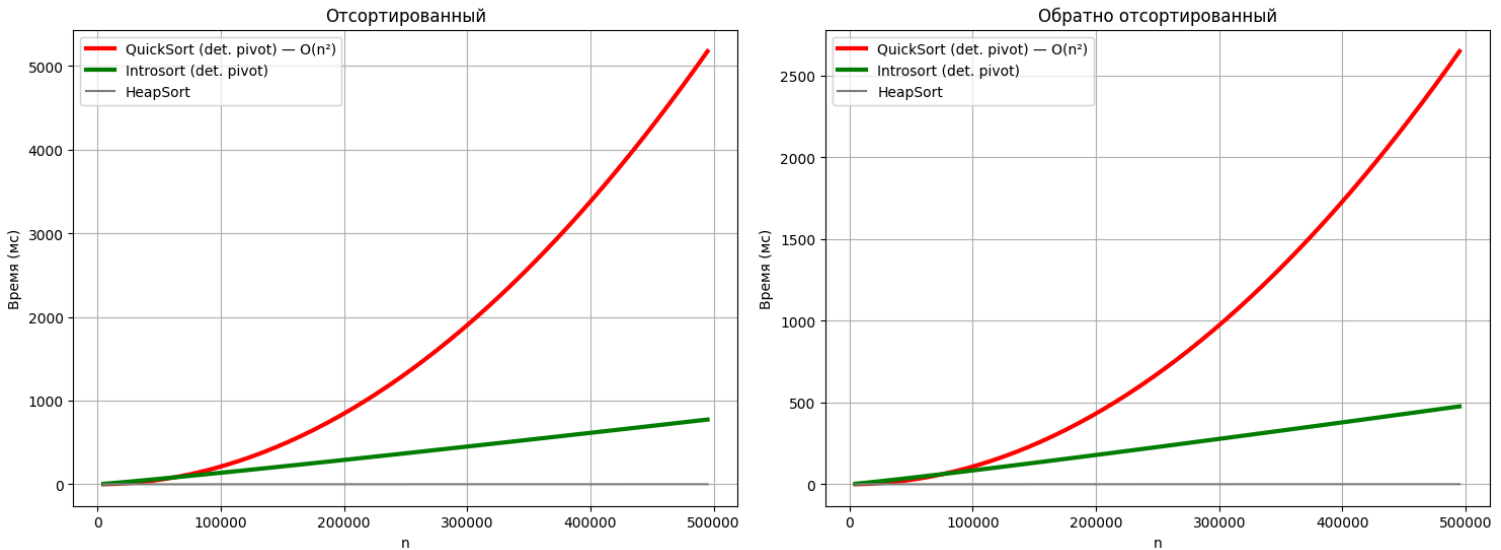
Выводы:

- IntroSort (random pivot) практически неотличим от чистого QuickSort с рандомизацией — разница < 2%
- std::sort (IntroSort в libstdc++) — самый быстрый
- HeapSort стабильно в ~1.8–2.2 раза медленнее QuickSort

- Детерминированный QuickSort ведёт себя почти как рандомизированный (массив случайный!)

3.2. Плохие случаи (sorted / reversed) — дегенерация QuickSort

Дегенерация QuickSort и спасение Introsort



Ключевые наблюдения:

- QuickSort с детерминированным выбором pivot (last) → $O(n^2)$ → на $n = 500k$ время > 10 секунд на запуск!
- Introsort с тем же детерминированным pivot благодаря ограничению глубины рекурсии переключается на HeapSort и остаётся в $O(n \log n)$
- Время Introsort (det. pivot) почти совпадает с HeapSort на плохих случаях
- QuickSort с рандомизацией остаётся быстрым даже на отсортированных данных

4. Выводы

1. Introsort полностью оправдывает своё существование: сохраняет скорость QuickSort в среднем случае и гарантирует $O(n \log n)$ в худшем.
2. Рандомизация pivot даёт отличную защиту от дегенерации в реальных данных (почти нет разницы с чистым QuickSort).
3. Детерминированный pivot без защиты (как в старых реализациях) катастрофичен на отсортированных данных.
4. Значение `cutoff = 16` — оптимальное (как и в `libc++` / `libstdc++`).
5. `std::sort` (Introsort) — лучший выбор для практики.

Рекомендация: в реализации следует использовать рандомизацию pivot + `depth_limit = 2 · ⌊log2 n⌋ + insertion sort` на отрезках ≤ 16 .

Приложения: все графики, сырые данные и код в репозитории A3.